Within each **GROUP**, certain operations are the same, and plots differ just by labeling so just a single figure is demonstrated for each group. However, complete MATLAB® codes for generation of all listed figures are to be found in Chapter 9.

Note that for averaged figures, **GROUP #3**, the averaging could be defined in different ways. As an initial choice, number of averages could be used, and this technique is shown in the current Chapter; however, generally it is recommended to use the user-defined frequency resolution instead, which is demonstrated in Chapter 9.

5.7.2.2 Time-Domain waveform (TD)

Time waveform is the most simple figure, which illustrates high-frequency data. The diagnostic value of original time waveform is frequently underestimated. In many cases of machine faults, its interpretation enables detection of cycles leading to fault detection and identification, without spectral analysis.

As in case of the trend plots, time waveform is displayed in MATLAB® using the ***plot*** command with two arguments, time and amplitude data. From MATLAB® point-of-view, the only difference between the trend plot and the time waveform plot, is that the first one is typically plotted with respect to an asynchronous time abscissa given in a separate data, whereas in case of time waveform, the values are plotted with respect to synchronous abscissa, which is generated from the sampling frequency and the total number of points. Following code shows a typical generation of a single time waveform plot:

```
x  = rand(1,250)-.5; % 250 random numbers
fs = 25e2;           % 2.5 kHz sampling frequency
N  = length(x);      % number of points (samples) in the signal
dt = 1/fs;           % constant time resolution
T  = N * dt;         % total signal time in [s]
t  = dt:dt:T;        % time vector

figure('Position',[300 300 550 200]); % all figures of the same size
plot(t,x,'b');               % generates figure
set(gca,'fontsize',8);       % set the size of the font
xlabel('Time [s]');          % labeling abscissa
ylabel('Value [m/s^2]');     % labeling ordinate
grid on;                     % sets additional grid
axis tight;                  % fills the graph with data
```

The resultant plot is shown in Figure 5.19.

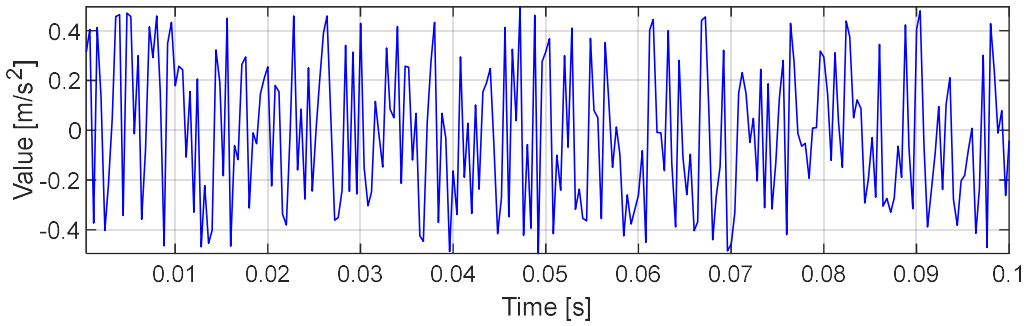

Figure 5.19 A typical time waveform

In commercial CMS, time waveforms typically have convenient cursor options, like harmonic cursors, as illustrated in Figure 5.20. The cursor corresponds to the dominant, low-frequency, 50 Hz component present in the signal.

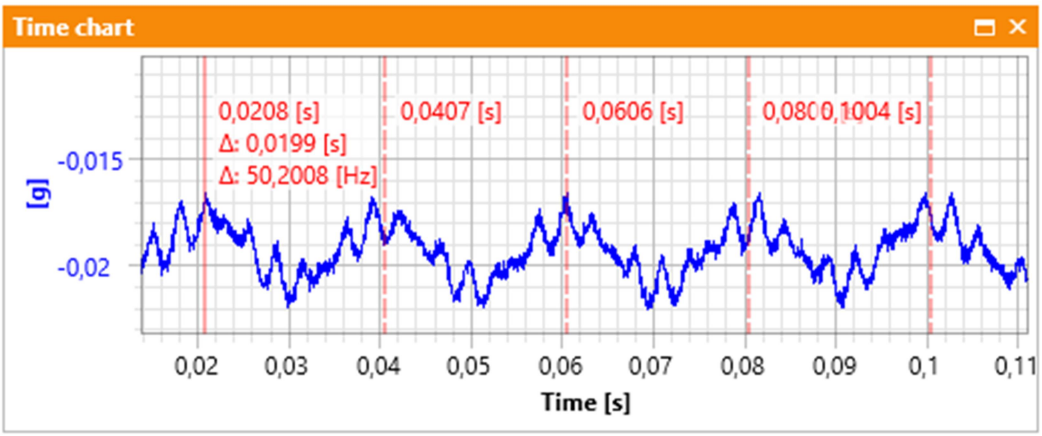

Figure 5.20 A typical time waveform with a harmonic cursor[Courtesy of AM Vibro™]

One of major drawbacks of time waveform is that its diagnostic value strongly depends on the current zoom (or the screen resolution), and capabilities of cursors arrangements.

NOTE: *Details of Envelope waveform (E) generation are given in Section 4.3.1. Details of Angle-Domain waveform (AD) generation are given in Section 4.3.1.8.*

5.7.2.3 Spectrum (S)

A full-resolution spectrum, often called (imprecisely, yet intuitively) the "FFT-spectrum", is one of most commonly used tool in the machine diagnostics, because it enables identification of signal frequency components. Following code shows how to generate a one-sided, scaled full resolution spectrum from a signal containing deterministic stationary and deterministic non-stationary components:

```
% SIGNAL PARAMETERS
T  = 0.5;    % total signal time
fs = 240;    % sampling frequency 240 Hz

% SIGNAL COMPONENTS
% deterministic components
A1 = 10; f1 = 20;
A2 = 13; f2 = 37;
A3 = 12; f3 = 46;

% non-stationary component
A_chirp = 17;
f_start = 60;
f_stop  = 80;

% GENERATE SIGNAL
dt = 1/fs;
t  = dt:dt:T;
N  = length(t);
s1 = A1*sin(2*pi*f1*t);
s2 = A2*sin(2*pi*f2*t);
s3 = A3*sin(2*pi*f3*t);
```

```
s4 = A_chirp*chirp(t,f_start,t(end),f_stop);
x  = s1 + s2 + s3 + s4;

% CALCULATE SPECTRAL AMPLITUDES
Y  = fft(x);      % Fast Fourier Transform (complex output)
Y  = abs(Y);      % spectral amplitudes - absolute value of complex numbers
Y  = Y(1:N/2+1); % one-sided spectrum, including DC and fs/2 components
Y  = Y/(N/2);     % scaling with respect to the number of samples
df = 1/T;         % spectral resolution, also: fs/N
f  = 0:df:fs/2;   % frequency vector

% PLOT FULL-RESOLUTION SPECTRUM
figure('Position',[300 300 550 200]); % all figures of the same size
plot(f,Y);
ylim([0 15])
set(gca,'fontsize',8);
title('Full resolution spectrum');
xlabel('Frequency [Hz]');
ylabel('Value [m/s^2]');
grid on;
```

Figure 5.21Illustrates the resultant plot.

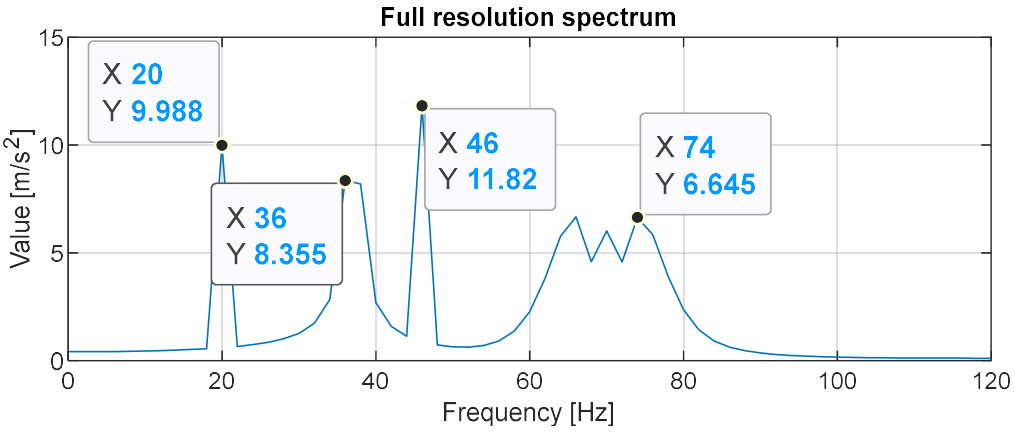

Figure 5.21 Scaled amplitudes of a full-resolution, one-sided frequency spectrum

Comparing the declared amplitudes of consecutive components (10, 13, 12, and 17 for chirp), Figure 5.21 clearly shows that for components, for which the frequency vector has “proper candidates”, the spectral components are generally accurately represented (first and third deterministic components), but in case, where the spectrum does not have a “proper candidate” (frequency 37Hz for the second deterministic component and the chirp component), the components are not represented accurately. This difference are a direct consequence of so-called “full cycles” of a particular component in the signal. These complete “cycles” are accessed by dividing the signal length by the cycle length of individual components, i.e.:

```
T/(1/f1) = T*f1 = 10;   % accurate   representation
T/(1/f2) = T*f2 = 18.5; % inaccurate representation
T/(1/f2) = T*f3 = 23;   % accurate   representation
```

Clearly, the second component with frequency equal to 37Hz has exactly 18 and a half periods, that is why the declared peak amplitude is significantly larger than the maximum spectral value of this component observed in Figure 5.21. However, the total power (i.e. the sum of square values) of all

points, which “contribute” to the representation of this component is still equal to the power of the declared component:

```
sum(s2.^2)/N = 84.5
```

From diagnostic-point of view, full-resolution spectrum has some disadvantages, mainly:

- For relatively large number of points, spectra are plotted using line-connected points, which implies a linear function between each pair of points, which is not true, because Fourier Transform is based on a set of orthogonal, i.e. independent functions [20]. Worth mentioning, historically, spectra were plotted using equivalent MATLAB® `stem` function.
- Because a spectrum is generated from independent functions, it must not show any dependencies between individual spectral component. In practice, this means that identification of any relations, like in the case of harmonics and sidebands, is always done as a separate analysis.
- Spectrum averages every time-variant components, which means that it is not capable to illustrate accurately any components, which are not stationary, like cyclostationary pulse responses generated by REB faults. This limitation is solved by so-called “Envelope Spectrum” (ES).
- Spectrum of a digital signal is always characterized by a fixed spectral resolution equal to the reciprocal of the signal length (or the number of revolutions for order domain); therefore, in practice, there is always a relatively large chance that a particular deterministic component (like GMF) is represented differently in each realization of the process, i.e**. in each spectrum of subsequent signa**l from the very same machine operating in the very same operational conditions. As a consequence, for consecutive spectra, neighboring spectral candidates might accept significantly different values (e.g. 30%), so the ratio of the amplitude (or power) of particular components changes as well. This is especially detrimental for full-resolution analysis of sideband-to-carrier ratio in the frequency domain. Referring to Figure 5.17, this drawback is reduced by either spectral averaging, signal resampling, or both.

Finally, from theoretical point-of-view, full-resolution spectrum is not designed for analysis of real vibration signals, because such signals are not perfectly deterministic and noise-free [20].

NOTE: *Details of Envelope Spectrum (ES) and Envelope Order Spectrum (EOS) generation are given in Section 4.3.1. Details of Order Spectrum (OS) generation are given in Section 4.2.2.*

5.7.2.4 Averaged Spectrum (AS)

Typically, the so-called “averaged spectrum” of a vibration signal is obtained by averaging a number of full-resolution spectra of individual fragments of the signal. Each fragment is of equal length in order to preserve a fixed frequency resolution. As a result, the final spectral resolution of the averaged spectrum is equal to the reciprocal of time **of a single fragment**. In terms of the number of samples, the length of the frequency scale of the averaged spectrum is equal to half of the number of samples of a single fragment plus one. On the other hand, the frequency range of the final averaged spectrum is **the same** as the frequency range of each fragment. For a one-sided spectrum, the frequency range spans from 0 to the half of sampling frequency. Following code shows a typical,

manual generation of a simplified averaged Spectrum (AS). The results of this function could be compared with spectral averaging functions available in MATLAB®, e.g. ***pwelch***:

```
function [AS] = Averaged_spectrum(data,k,overlap,win)
% DESCRIPTION
% Function calculates scaled values of one-sided averaged spectrum of the
% signal "x" using the number of averages specified by "k" and the overlap
% specified by "overlap". The "win" parameter enables application of the
% hanning window to each fragment.

% INPUT
% data      - a single time signal to be processed, size <1xN>
% k         - integer number of averages
% overlap   - user overlap [%], recommended: (0-90)
% win       - use "hanning" for the hanning window; no window otherwise

% OUTPUT
% AS          - scaled amplitudes of averaged one-sided spectrum

% EXAMPLE
% AS = Averaged_spectrum(x,10);            % simple average of 10 fragments

if nargin < 4
    win = 'rect';       % default window is rectangular (no windowing)
    if nargin < 3
        overlap = 0;    % default overlap is no overlap
    end
end

% SIGNAL PRE-PROCESSING
N         = length(data);                     % No. of samples in the signal
w_len     = floor(N/(k-overlap*(k-1)/100));   % No. of samples in a fragment

if mod(w_len,2)
    w_len = w_len - 1;                        % w_len must be an even number
end

N_overlap = floor(w_len*overlap/100);         % No. of overlapping samples
FFT_array = zeros(k, w_len/2+1);              % allocate memory

if strcmpi(win,'hanning')
    w      = .5*(1 - cos(2*pi*(1:w_len)'/(w_len+1))); % hanning window
    w_coef = 1.4;                                     % window compensation
elseif strcmpi(win,'rect')
    w      = ones(w_len,1);                           % rectangular window
    w_coef = 1;                                       % window compensation
end

% SIGNAL PROCESSING
for i = 1:k                           % for every fragment of the signal

    disp(['Processing fragment No.: ',num2str(i),' ...'])
    index_from = (w_len - N_overlap)*(i-1) + 1;
    index_till =  index_from + w_len - 1;

    x = data(index_from:index_till); % current fragment
    x = x(:).*w;                     % windowing, x always a column vector
    Y = abs(fft(x));                 % spectral amplitudes
    Y = Y(1:w_len/2+1)/(w_len/2);    % extracting one side and scaling
```

```
    Y = Y.*w_coef;                  % approximated windowing compensation
    FFT_array(i,:) = Y;             % current row of the final array

end

disp(['Last ',num2str(N-index_till),' samples of the signal truncated.']);
AS = mean(FFT_array,1);             % average spectrum
disp('Processing finished. ')
```

Worth mentioning, a literal meaning of the "**Averaged Spectrum**" is not accurate, because it implies arithmetical averaging of spectral values of a full-resolution spectrum in subsequent "bins". Although for majority of cases, such averaging yields similar results as the arithmetical averaging of spectral values calculated from individual fragment of the signal, it is not mathematically meaningful, because the spectral values within such bin are independent, while averaging refers to results of some function. Moreover, averaging of spectral amplitudes does not allow any overlapping.

In practical vibration-based analysis, **Averaged Spectrum (AS)** is frequently identified either with "**PSD**", "**(averaged) periodogram**", or with "**pwelch**" spectrum, which cover both, precise signal processing method as well as particular way of data plotting (algorithm + figure). **PSD** (from Power Spectrum Density) implies that signal **power** is analyzed, so **square** of spectral averaged values is plotted. **Periodogram** is a more general term than **PSD**, because it is not limited to signal "power", but it includes the **PSD** as one of its realization. **Pwelch** is a the most defined term, because it suggests particular signal segmentation, overlap, and windowing of individual fragments. Because any **periodogram** realization reduces spectral noise, typically averaged spectra are used for broadband comparison in logarithmic (dB) scale.

NOTE:

- *Details of (resolution-based) Averaged Spectrum (AS) generation are given in Section 9.5.3.5*
- *Details of Averaged Envelope Spectrum (AES) generation are given in Section 9.5.3.5*
- *Details of Averaged Order Spectrum (AOS) generation are given in Section 9.5.3.5*
- *Details of Averaged Envelope Order Spectrum (AEOS) generation are given in Section 9.5.3.5*
- *Details of TSA-based spectra generation are given in Section 4.3.3*

Next Section describes generation of selected 3-dimensional figures for a single vibration waveform.

5.7.3 3-D figures generated from a single signal (colormaps)

Recalling, a large list of 3-dimensional figures, which are used in vibration-based analysis are illustrated in Figure 5.17. For some figures, the codes are commercially available, for other they are free, yet for some just the analytical description is given in scientific publications. The current Section lists the codes, which are easily found.

Spectrogram

Spectrogram is one of the most popular time-frequency analysis used for analysis of non-stationary signals. Sometimes, spectrogram helps to identify high-frequency pulses, as well. Finally, spectrogram is especially useful to illustrate multiple frequency modulations of various kind on a single plot. A user-friendly code for spectrogram analysis is available directly by MATLAB® `spectrogram` function. Additionally, MathWorks® *File Exchange Community* offers numerous implementations of live spectrogram data processing.